

Spencer Collins

Assignment 5

06/18/2026

5.1: Why do we need to do the putchar loop in bytestring_print_ascii? Why can't we replace the entire loop:

```
for (int i = 0; i < b->length; i++) {  
    putchar(b->data[i]);  
}
```

with

```
printf("%s", b->data)
```

? More specifically, what input might cause the latter to fail?

You need to use putchar() in a loop because a string in C is really a character array where each character is an element in that array. Trying to call putchar() like in the second method does not work for print character arrays with more than one element. It has to be printed element by element to stdout. If "b->data" is a character array with one element, putchar() will work.

5.2: Consider an alternate data structure to represent dyn_value:

```
const int DBool = 0;  
const int DNumber = 1;  
const int DString = 2; // same as above  
  
typedef struct dyn_value_2 {  
    int    type;  
    bool   as_bool;  
    double as_number;  
    bytestring* as_string;  
} dyn_value_2;
```

How would one use this data structure, instead of the void* based one? Is it legal to do this? What advantages does it have? What disadvantages does it have?

By defining a struct as typedef for dyn_value_2, you can encapsulate/store all the properties of dyn_value_2 inside one container and you can type "dyn_value_2 var_name" instead of having to write "struct dyn_value_2 var_name" every time you initialize a new dyn_value_2 variable. You can now have as many dyn_value_2 variables

as you want with different names and don't have to initialize three new variables for every `dyn_value_2` variable that gets created. A disadvantage is now the developer has to remember what the underlying data type otherwise an incorrect data type may be passed into a function and the compiler may not catch it. It is legal to create the second data structure.

5.3: Run your StackMan implementation on the six tests (`test_1.sm`, ..., `test_6.sm`) supplied in Canvas under Files. Some of these will terminate normally; others will generate errors (i.e., they are not valid StackMan programs when executed against an empty stack.) None of them loop indefinitely (although that is possible in StackMan.) In the case of successful execution, tell me what the stack contents are after successful execution as well as the instruction count. For example:

```
$ ./stackman test_1.sm
# instructions: 40
final stack contents: 2 0 2 5
```

In an error case, tell me which error is produced.

For `test_5.sm` and `test_6.sm`, I also want you to tell me how long your implementation takes. You are not graded (so long as your programs do terminate) on performance; the lesson here is to observe and report. Use the Unix time utility or something similar if available; if not, stopwatch time is acceptable.

5.4: Include your StackMan implementation code on subsequent pages.

Stackman Outputs:

```
PS C:\Users\spenc\OneDrive\Desktop\GA Tech Online CS Master's Degree\CS8001-OIC_Intro_to_C_Programming\Assignment_5> gcc -o stackman stackman.c
PS C:\Users\spenc\OneDrive\Desktop\GA Tech Online CS Master's Degree\CS8001-OIC_Intro_to_C_Programming\Assignment_5> ./stackman test_1.sm
# instructions: 40
final stack contents: 2 0 2 5
time to run: 0.000230 seconds
PS C:\Users\spenc\OneDrive\Desktop\GA Tech Online CS Master's Degree\CS8001-OIC_Intro_to_C_Programming\Assignment_5> ./stackman test_2.sm
Error: stack underflow
PS C:\Users\spenc\OneDrive\Desktop\GA Tech Online CS Master's Degree\CS8001-OIC_Intro_to_C_Programming\Assignment_5> ./stackman test_3.sm
# instructions: 197
final stack contents: 4
time to run: 0.000214 seconds
PS C:\Users\spenc\OneDrive\Desktop\GA Tech Online CS Master's Degree\CS8001-OIC_Intro_to_C_Programming\Assignment_5> ./stackman test_4.sm
Error: stack overflow
PS C:\Users\spenc\OneDrive\Desktop\GA Tech Online CS Master's Degree\CS8001-OIC_Intro_to_C_Programming\Assignment_5> ./stackman test_5.sm
# instructions: 6237
final stack contents: 139
time to run: 0.000232 seconds
PS C:\Users\spenc\OneDrive\Desktop\GA Tech Online CS Master's Degree\CS8001-OIC_Intro_to_C_Programming\Assignment_5> ./stackman test_6.sm
# instructions: 178477007
final stack contents: 743
time to run: 0.346371 seconds
```

Stackman Code:

```
#include <stdio.h>

#include <stdlib.h>
```

```
#include <string.h>

#include <time.h>

void print_stack(int tos_idx, int stack[65536]);

//Only the characters 0, +, <, >, ?, _, ~, (, ) are valid. Treat all others as
comments--ignore them.

int main(int argc, char* argv[]) {

    struct timespec start;

    clock_gettime(CLOCK_MONOTONIC, &start);

    if (argc > 2) {

        printf("Error: Too many input arguments.");

        return 1;

    }

    if (argv[1] == "") {

        printf("Error: No filename was provided.");

        return 1;

    }
```

```
const char* idx = strstr(argv[1], ".sm");

if (idx == NULL) {
    printf("Error: File extension is not .sm, unable to open file.");
    return 1;
}

FILE* file;
char* filename = argv[1];

file = fopen(filename, "r");

if (file == NULL) {
    printf("Error: File does not exist or unable to open file.");
    return 1;
}

char line[1024];

fgets(line, sizeof(line), file);
```

```
fclose(file);

int ops[4096];

int op_jump_table[4096] = {0};
int cp_jump_table[4096] = {0};

int indx = 0;
for (int i=0; i<sizeof(line);i++) {
    if (line[i] == '(') {
        ops[indx] = i;
        indx++;
    } else if (line[i] == ')') {
        op_jump_table[ops[indx-1]] = i;
        cp_jump_table[i] = ops[indx-1];
        indx--;
    } else if (line[i] == '\\0') {
        break;
    }
}

if (indx > 0) {
    printf("Error: syntax error, mismatching parentheses");
    return 1;
}
```

```
}
```

```
int stack[65536];
```

```
int tos_idx = -1;
```

```
int num_commands = 0;
```

```
int i = 0;
```

```
while (i<sizeof(line)) {
```

```
    char command = line[i];
```

```
    if (tos_idx > 65536) {
```

```
        printf("Error: stack overflow");
```

```
        return 1;
```

```
    }
```

```
int eol = 0;
```

```
switch (command) {
```

```
    default:
```

```
        printf("Error: syntax error, invalid character found");
```

```
        return 1;
```

```
case '\\0':
```

```
    eol = 1;
```

```
    break;
```

```
case '0':
```

```
    tos_idx++;
```

```
    stack[tos_idx] = 0;
```

```
    break;
```

```
case '+':
```

```
    if (tos_idx == -1) {
```

```
        printf("Error: stack underflow");
```

```
        return 1;
```

```
    }
```

```
    stack[tos_idx]++;
```

```
    break;
```

```
case '>':
```

```
if (tos_idx == -1) {  
    printf("Error: stack underflow");  
    return 1;  
}  
  
int n = stack[tos_idx];  
  
if (n < 2 || n > tos_idx) {  
    printf("Error: stack underflow");  
    return 1;  
}  
  
int end = tos_idx;  
int start = end - n;  
  
int temp = stack[end-1];  
for (int i = end-1; i>start; i--) {  
    stack[i] = stack[i-1];  
}  
stack[start] = temp;  
tos_idx--;  
break;
```



```
case '<': {
    if (tos_idx == -1) {
        printf("Error: stack underflow");
        return 1;
    }

    int n = stack[tos_idx];

    if (n < 2 || n > tos_idx) {
        printf("Error: stack underflow");
        return 1;
    }

    int end = tos_idx;
    int start = end - n;

    int temp = stack[start];

    for (int i = start; i < end-1; i++) {
        stack[i] = stack[i+1];
    }
    stack[end-1] = temp;
    tos_idx--;
    break;
}
```

```
case '?': {

    if (tos_idx == -1) {
        printf("Error: stack underflow");
        return 1;
    }

    if (stack[tos_idx] == 0) {
        tos_idx++;
        stack[tos_idx] = 0;
    } else if (stack[tos_idx] > 0) {
        stack[tos_idx]--;
        tos_idx++;
        stack[tos_idx] = 1;
    }

    break;
}

case '_':

    if (tos_idx == -1) {
        printf("Error: stack underflow");
        return 1;
    }

    tos_idx--;

    break;
```

```
case '~':

    if (tos_idx == -1) {
        printf("Error: stack underflow");
        return 1;
    }

    int dup = stack[tos_idx];
    tos_idx++;
    stack[tos_idx] = dup;
    break;
case '(':
    if (stack[tos_idx] == 0) {
        i = op_jump_table[i];
    }
    break;
case ')':
    if (stack[tos_idx] != 0) {
        i = cp_jump_table[i];
    }
    break;
}

if (eol == 1) {
    break;
}

num_commands++;
```

```

        i++;

    }

    printf("# instructions: %d\n", num_commands);
    printf("final stack contents: ");
    print_stack(tos_idx, stack);

    struct timespec end;
    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec -
start.tv_nsec) / 1000000000.0;

    printf("\ntime to run: %lf seconds", elapsed_time);

    return 0;
};

void print_stack(int tos_idx, int stack[65536]) {
    if (tos_idx == -1) {
        printf("empty stack");
    } else {
        for (int i=0; i<=tos_idx; i++) {

```

```
        printf("%d ", stack[i]);  
    }  
}  
return;  
}
```